# Statistical Tests of Regression: Simple Linear Regression

**v1.0.0**

Allan Omondi

25 June 2026

## Table of contents

# 1 Statistical Tests of Regression: Simple Linear Regression

## 1.1 What is Simple Linear Regression?

**Simple linear regression** is a statistical technique used to model the relationship between:

- **one independent variable** `X` (predictor / explanatory variable), and
- **one dependent variable** `Y` (response / outcome variable).

Its purpose is to determine whether a **linear relationship** exists between the two variables and, if so, to estimate that relationship with a straight-line equation.

In this lesson:

- **Independent variable** `X`: `purchase_frequency`

- **Dependent variable** `Y`: `customer_lifetime_value`
- This means we want to examine whether customers who purchase more frequently tend to have a higher estimated customer lifetime value.

## 1.2 The Regression Equation

Simple linear regression fits a line of the form:

$Y = \beta_0 + \beta_1 X + \epsilon$

Where:

- `Y` = dependent variable (it depends on the value of `X`)
- `X` = independent variable
- $\beta_0$ = **intercept**
  - the predicted value of `Y` when `X = 0`
- $\beta_1$ = **slope**
  - the expected change in `Y` for a one-unit increase in `X`
- $\epsilon$ = random error term

In practice, because we work with sample data, we estimate the model as:

$\hat{Y} = b_0 + b_1 X + \epsilon$

where $b_0$ and $b_1$ are the estimated regression coefficients from the sample.

## 1.3 Context for This Lab

In this lab, we use simple linear regression to model the relationship between:

- **Purchase Frequency** $\rightarrow$ predictor
- **Customer Lifetime Value (CLV)** $\rightarrow$ response

We investigate whether purchase frequency is a useful predictor of customer lifetime value and interpret the fitted regression model in business terms.

The regression model helps answer the question:

**How does purchase frequency affect customer lifetime value? or**

**How much does customer lifetime value change when purchase frequency increases by 1?**

If the slope $b_1$ is positive, it suggests that customers who buy more often tend to have higher lifetime value. If it is negative, it would suggest the opposite.

## 1.4 Why Use Simple Linear Regression?

Simple linear regression is useful for two main reasons:

### 1.4.1 1. Explanation

It helps us understand the **direction and strength** of the relationship between two variables.

For example:

- Is the relationship positive or negative?
- Is the relationship weak or strong?
- Is the predictor statistically significant?

### 1.4.2 2. Prediction

It allows us to **predict** the dependent variable from the independent variable.

For example:

- given a customer's purchase frequency, estimate their expected customer lifetime value.

## 1.5 Key Outputs to Interpret

When fitting a simple linear regression model, the most important outputs are:

### 1.5.1 1. Intercept ( $b_0$ )

The estimated value of `Y` when `X = 0`.

#### 1.5.2 2. Slope ( $b_1$ )

Shows how much `Y` is expected to change for a one-unit increase in `X`.

#### 1.5.3 3. $R^2$ (coefficient of determination)

Shows the proportion of variation in the dependent variable explained by the model.

- $R^2 = 0$ : the model explains none of the variation
- $R^2 = 1$ : the model explains all of the variation

#### 1.5.4 4. p-value for the slope

Tests whether the independent variable (predictor) has a statistically significant linear relationship with the dependent (response) variable.

A common hypothesis test is:

- $H_0 : \beta_1 = 0$
- $H_1 : \beta_1 \neq 0$

If the p-value is small (for example, less than 0.05), we reject $H_0$ (the null hypothesis) and conclude that the predictor is significantly associated with the response variable.

## 2 Install Dependencies

```
# `installed.packages()` retrieves a matrix of all installed packages
# `[, "Package"]` extracts on the "Package" column from the matrix of all
# packages. This contains the name of the package.
# The %in% operator is used to test if the specified package is in the matrix of
# all packages
# `dependencies = TRUE` instructs R to install not only the specified package but
# also its dependencies

if (!"pacman" %in% installed.packages()[, "Package"]) {
  install.packages("pacman", dependencies = TRUE)
  library("pacman")
}
```

```
# For reading data from a CSV file
pacman::p_load("readr")
# For data visualization
pacman::p_load("ggplot2")
# For creating a correlation plot using ggplot2
pacman::p_load("ggcorrplot")
# For creating a matrix of plots using ggplot2
pacman::p_load("GGally")
# For various statistical functions
pacman::p_load("e1071")
# For visualizing missing (Not Applicable) data
pacman::p_load("naniar")
# For reshaping data required by the correlation plot
pacman::p_load("reshape2")
```

## 3 Load the Dataset

The following synthetic dataset contains the estimated Customer Lifetime Value (CLV) as the dependent variable and the customer purchase frequency as the independent variable. The dataset is loaded as shown below.

```
clv_data <- read_csv("./data/clv_data.csv")
head(clv_data)
```

```
#> # A tibble: 6 x 2
#>   purchase_frequency customer_lifetime_value
#>                <dbl>                   <dbl>
#> 1                  3                   110.
#> 2                  7                   190.
#> 3                  6                   160.
#> 4                  2                    94.4
#> 5                  4                   133.
#> 6                  8                   223.
```

## 4 Initial EDA

### View the Dimensions

The number of observations and variables.

```r
dim(clv_data)
```

```
#> [1] 500   2
```

### View the Data Types

```r
# sapply() is designed to apply a function to a variable in a dataset
# Example: `sapply(clv_data, class)` applies the `class()` function to each
# variable in the `clv_data` dataset, returning the data type of each variable.

sapply(clv_data, class)
```

```
#>      purchase_frequency customer_lifetime_value
#>               "numeric"               "numeric"
```

```r
str(clv_data)
```

```
#> spc_tbl_ [500 x 2] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
#>  $ purchase_frequency     : num [1:500] 3 7 6 2 4 8 0 4 8 3 ...
#>  $ customer_lifetime_value: num [1:500] 110.3 190.2 160 94.4 133.2 ...
#>  - attr(*, "spec")=
#>   .. cols(
#>   ..   purchase_frequency = col_double(),
#>   ..   customer_lifetime_value = col_double()
#>   .. )
#>  - attr(*, "problems")=<externalptr>
```

### Descriptive Statistics

Descriptive statistics can be used to understand your data. Understanding your data can lead to:

- Verifying data-entry accuracy, investigating unusual cases, deleting observations with outliers or imputing missing data.
- **Data transformation:** Using techniques such as scaling (dividing each value by the standard deviation), centering (subtracting the mean from each value), standardization (ensuring each numeric attribute has a mean of 0 and a standard deviation of 1), normalization (rescaling the values to a range of between 0 and 1 (inclusive)), reducing skewness using Box-Cox or Yeo-Johnson transformations.

- **Hypothesis formulation:** Formulate a hypothesis based on the patterns you identify.
- **Choosing the appropriate statistical test:** You may notice properties of the data such as distributions or data types that suggest the use of parametric or non-parametric statistical tests and parametric (linear) or non-parametric (non-linear) algorithms.

Typical descriptive statistics include:

1. **Measures of frequency:** count and percent
2. **Measures of central tendency:** mean, median, and mode
3. **Measures of distribution/dispersion/spread/scatter/variability:** minimum, quartiles, maximum, variance, standard deviation, coefficient of variation, range, interquartile range (IQR) [includes a box and whisker plot for visualization], kurtosis, skewness [includes a histogram for visualization]).
4. **Measures of relationship:** covariance and correlation

## 4.1 Measures of Frequency — Not Applicable

This is applicable in cases where you have categorical variables, e.g., 60% of the observations are male and 40% are female (2 categories for the gender). This informs you about class imbalance.

## 4.2 Measures of Central Tendency

The median and the mean of each numeric variable:

```
summary(clv_data)
```

```
#>  purchase_frequency customer_lifetime_value
#>  Min.   :-1.000     Min.   : 26.13
#>  1st Qu.: 4.000     1st Qu.:122.04
#>  Median : 5.000     Median :148.21
#>  Mean   : 4.914     Mean   :148.25
#>  3rd Qu.: 6.000     3rd Qu.:175.88
#>  Max.   :11.000     Max.   :262.04
```

The first 5 observations (rows) in the dataset:

```r
head(clv_data, 5)
```

```
#> # A tibble: 5 x 2
#>   purchase_frequency customer_lifetime_value
#>                <dbl>                   <dbl>
#> 1                  3                   110.
#> 2                  7                   190.
#> 3                  6                   160.
#> 4                  2                    94.4
#> 5                  4                   133.
```

The last 5 observations (rows) in the dataset:

```r
tail(clv_data, 5)
```

```
#> # A tibble: 5 x 2
#>   purchase_frequency customer_lifetime_value
#>                <dbl>                   <dbl>
#> 1                  6                   176.
#> 2                  6                   176.
#> 3                  5                   144.
#> 4                  6                   165.
#> 5                  5                   139.
```

## 4.3 Measures of Distribution

Measuring the variability in the dataset is important because the amount of variability determines **how well you can generalize** results from the sample to a new observation in the population.

Low variability is ideal because it means that you can better predict information about the population based on the sample data. High variability means that the values are less consistent, thus making it harder to make predictions.

The syntax `dataset[rows, columns]` can be used to specify the exact rows and columns to be considered. `dataset[, columns]` implies all rows will be considered. For example, specifying `BostonHousing[, -4]` implies all the columns except column number 4. This can also be stated as `BostonHousing[, c(1,2,3,5,6,7,8,9,10,11,12,13,14)]`. This allows us to perform calculations on only columns that are numeric, thus leaving out the columns termed as "factors" (categorical) or those that have a string data type.

### 4.3.1 Variance

```
sapply(clv_data[,], var)
```

```
#>      purchase_frequency customer_lifetime_value
#>                4.146898             1642.315996
```

### 4.3.2 Standard Deviation

```
sapply(clv_data[,], sd)
```

```
#>      purchase_frequency customer_lifetime_value
#>                2.036393               40.525498
```

### 4.3.3 Kurtosis (Pearson)

The Kurtosis informs us of how often outliers occur in the results. There are different formulas for calculating kurtosis. Specifying "type = 2" allows us to use the 2nd formula which is the same kurtosis formula used in other statistical software like SPSS and SAS. It is referred to as "Pearson's definition of kurtosis".

In "type = 2" (used in SPSS and SAS):

1. Kurtosis $< 3$ implies a low number of outliers $\rightarrow$ platykurtic
2. Kurtosis $= 3$ implies a medium number of outliers $\rightarrow$ mesokurtic
3. Kurtosis $> 3$ implies a high number of outliers $\rightarrow$ leptokurtic

High kurtosis (leptokurtic) affects models that are sensitive to outliers. Estimates of the variance are also inflated. Low kurtosis (platykurtic) implies a possible underestimation of real-world variability. The typical remedy includes trimming outliers or using robust statistical methods that are less affected by outliers.

```
sapply(clv_data[,],  e1071::kurtosis, type = 2)
```

```
#>      purchase_frequency customer_lifetime_value
#>              -0.1220038              -0.1484811
```

### 4.3.4 Skewness

The skewness is used to identify the asymmetry of the distribution of results. Similar to kurtosis, there are several ways of computing the skewness.

Using "type = 2" (common in other statistical software like SPSS and SAS) can be interpreted as:

1. Skewness between -0.4 and 0.4 (inclusive) implies that there is no skew in the distribution of results; the distribution of results is symmetrical; it is a normal distribution; a Gaussian distribution.
2. Skewness above 0.4 implies a positive skew; a right-skewed distribution.
3. Skewness below -0.4 implies a negative skew; a left-skewed distribution.

Skewed data results in misleading averages and potentially biased model coefficients. The typical remedy to skewed data involves applying data transformations such as Box–Cox or Yeo-Johnson transformations to reduce skewness.

```
sapply(clv_data[,], e1071::skewness, type = 2)
```

```
#>      purchase_frequency customer_lifetime_value
#>             -0.04021915             -0.01608242
```

**NOTE:** As a data analyst, you need to confirm if the distortion in kurtosis or skewness is a data problem or it is a real-world insight. For example, a real-world insight could be that few customers drive most of the value. This is as opposed to always looking it at it as a distortion that needs to be corrected.

## 4.4 Measures of Relationship

### 4.4.1 Covariance

Covariance is a statistical measure that indicates the direction of the linear relationship between two variables. It assesses whether increases in one variable correspond to increases or decreases in another.

- **Positive Covariance:** When one variable increases, the other tends to increase as well.
- **Negative Covariance:** When one variable increases, the other tends to decrease.
- **Zero Covariance:** No linear relationship exists between the variables.

While covariance indicates the direction of a relationship, it does not convey the strength or consistency of the relationship. The correlation coefficient is used to indicate the strength of the relationship.

```
cov(clv_data, method = "spearman")
```

```
#>                          purchase_frequency customer_lifetime_value
#> purchase_frequency                20409.91                20235.73
#> customer_lifetime_value           20235.73                20874.99
```

### 4.4.2 Correlation

A strong correlation between variables enables us to better predict the value of the dependent variable using the value of the independent variable. However, a weak correlation between two variables does not help us to predict the value of the dependent variable from the value of the independent variable. This is useful only if there is a linear association between the variables.

We can measure the statistical significance of the correlation using Spearman's rank correlation *rho*. This shows us if the variables are significantly monotonically related. A monotonic relationship between two variables implies that as one variable increases, the other variable either consistently increases or consistently decreases. The key characteristic is the preservation of the direction of change, though the rate of change may vary.

```
cor.test(clv_data$customer_lifetime_value, clv_data$purchase_frequency, method = "spearman")
```

```
#>
#>  Spearman's rank correlation rho
#>
#> data:  clv_data$customer_lifetime_value and clv_data$purchase_frequency
#> S = 409190, p-value < 2.2e-16
#> alternative hypothesis: true rho is not equal to 0
#> sample estimates:
#>       rho
#> 0.9803588
```

To view the correlation of all variables

```
cor(clv_data, method = "spearman")
```

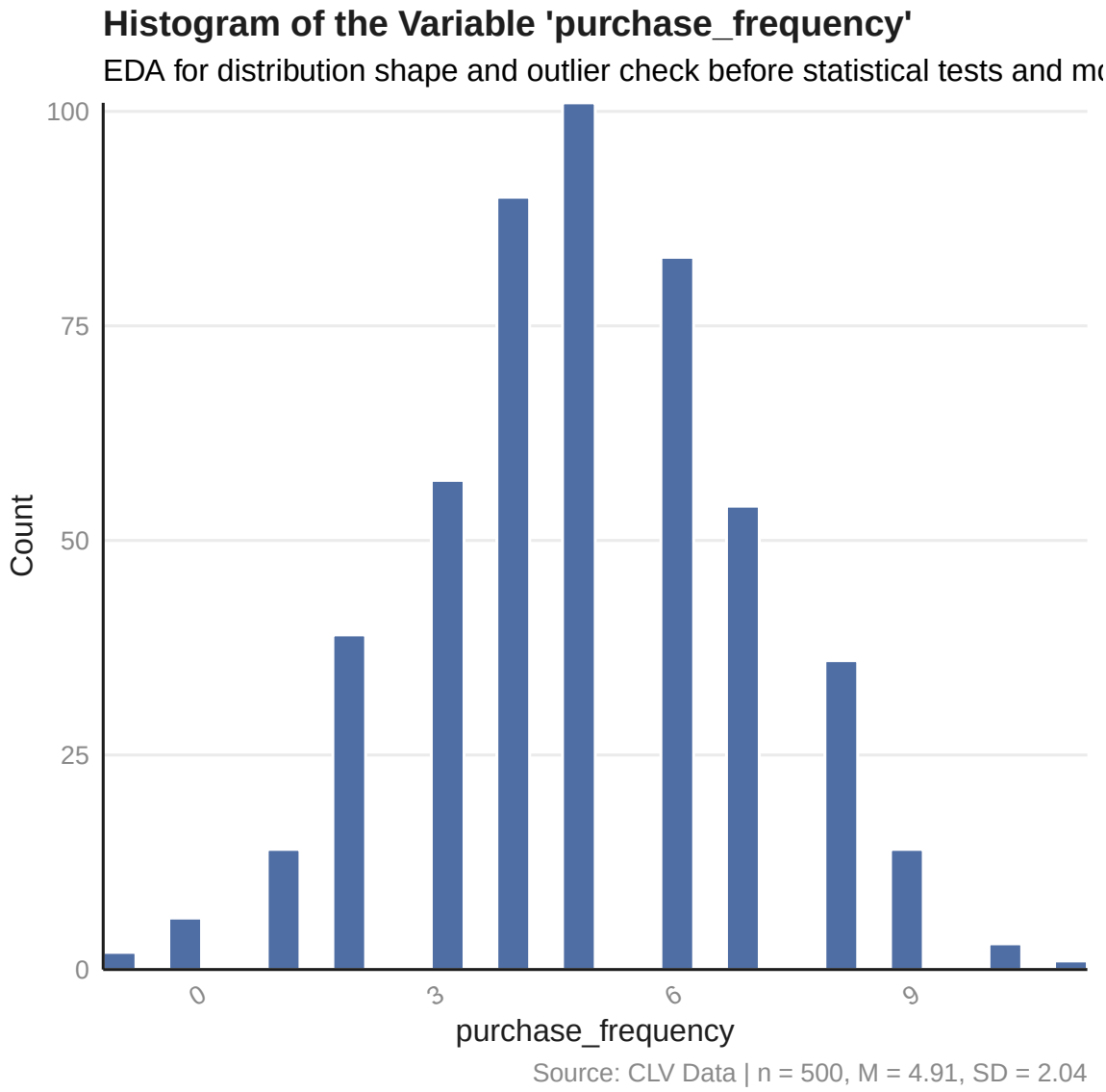
```
#>                          purchase_frequency customer_lifetime_value
#> purchase_frequency               1.0000000               0.9803588
#> customer_lifetime_value          0.9803588               1.0000000
```

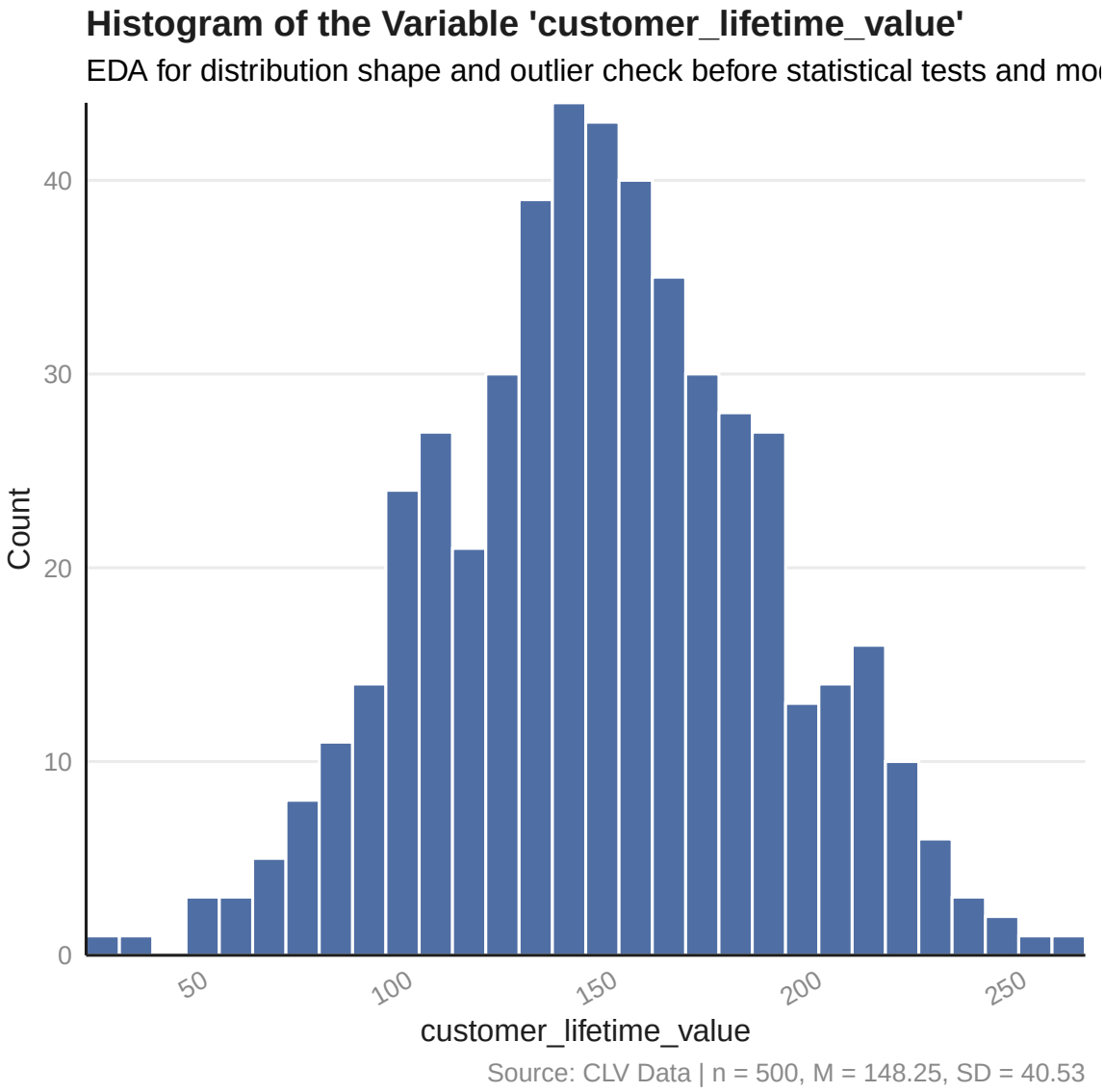
## 4.5 Basic Visualizations

### 4.5.1 Histogram

```
# `col_index` identifies which column(s) (by position) are being plotted;
# seq_along() generates 1, 2, ..., ncol(data_frame) automatically,
# so every column in the data frame is covered without manually listing
# positions
col_index <- seq_along(clv_data)
# the right-hand `col_index` (the full vector) is evaluated once when the
# loop starts; col_index is then reassigned to one position per iteration.
for (col_index in col_index) {

  # skip silently, with no message/warning, if the position does not exist
  # in the data (out of range) or if the column at that position is not
  # numeric; `next` moves straight to the following value of col_index
  if (col_index > ncol(clv_data) ||
      !is.numeric(clv_data[[col_index]])) {
    next
  }

  col_name <- names(clv_data)[col_index]

  # for a histogram, the continuous variable goes on x; geom_histogram()
  # bins it and computes the count on y automatically, so y is left
  # unmapped here
  p <- ggplot2::ggplot(clv_data, aes(x = .data[[col_name]])) +
    geom_histogram(bins = 30, fill = "#4F6EA4", color = "#FFFFFF") +

    # reduces the default 5% padding on the x and y axes
    scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
    scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +
    labs(
      title = paste0("Histogram of the Variable '", col_name, "'"),
      subtitle = paste0("EDA for distribution shape and outlier check ",
                        "before statistical tests and modelling"),
      x = col_name,
      y = "Count",
      caption = paste0("Source: CLV Data | n = ",
                       nrow(clv_data), ", ",
                       "M = ",
```

```
                        sprintf("%.2f",
                                mean(clv_data[[col_name]], na.rm = TRUE)),
                        ", ",
                        "SD = ",
                        sprintf("%.2f",
                                sd(clv_data[[col_name]], na.rm = TRUE)))
    ) +
    theme_minimal(base_size = 12) +

    theme(
      plot.title          = element_text(face = "bold", color = "#1D1D1D"),
      axis.title          = element_text(color = "#1D1D1D"),
      axis.text           = element_text(color = "#898989"),
      axis.text.x         = element_text(angle = 30, hjust = 1),
      plot.caption         = element_text(color = "#898989"),
      # removes vertical gridlines
      panel.grid.major.x  = element_blank(),
      panel.grid.minor.x  = element_blank(),
      # horizontal gridlines are kept (at major breaks only) since they
      # help the reader judge bar heights against the y-axis scale
      # This is part of maintaining a high data-to-ink ratio in your
      # visualizations as discussed during the theory class.
      # panel.grid.major.y  = element_blank(),
      panel.grid.minor.y  = element_blank(),
      # draws only the x-axis and y-axis lines (the "L" frame)
      axis.line.x         = element_line(color = "#1D1D1D"),
      axis.line.y         = element_line(color = "#1D1D1D")
      # panel.border = element_rect(color = "#1D1D1D", fill = NA,
      #                             linewidth = 0.5)
    )
  # ggplot objects do not auto-print inside a for-loop, therefore,
  # each one must be printed explicitly otherwise it will never appear
  print(p)
}
```


Histogram of the Variable 'purchase_frequency'
EDA for distribution shape and outlier check before statistical tests and mo
100
75
50
25
0
Count
0
3
6
9
purchase_frequency
Source: CLV Data | n = 500, M = 4.91, SD = 2.04



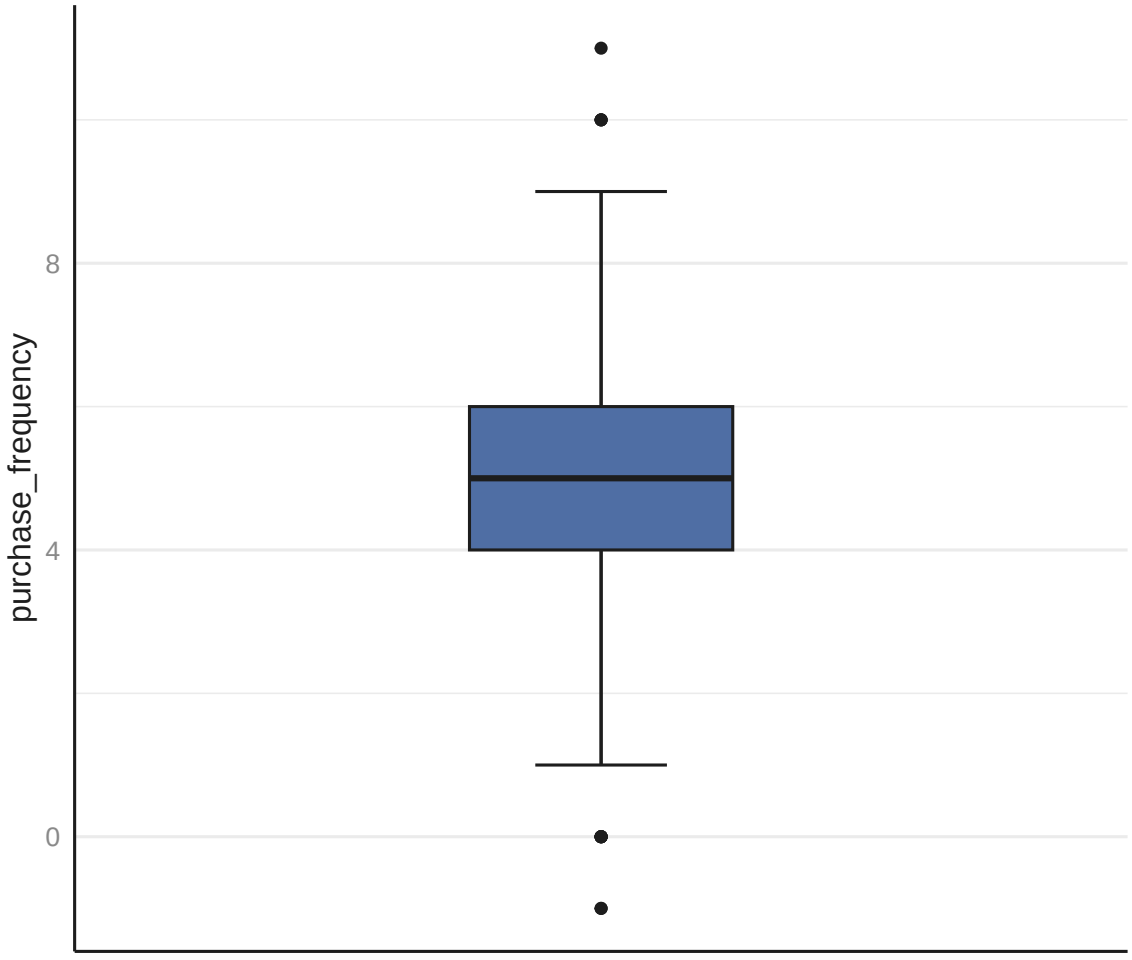

### 4.5.2 Box and Whisker Plot

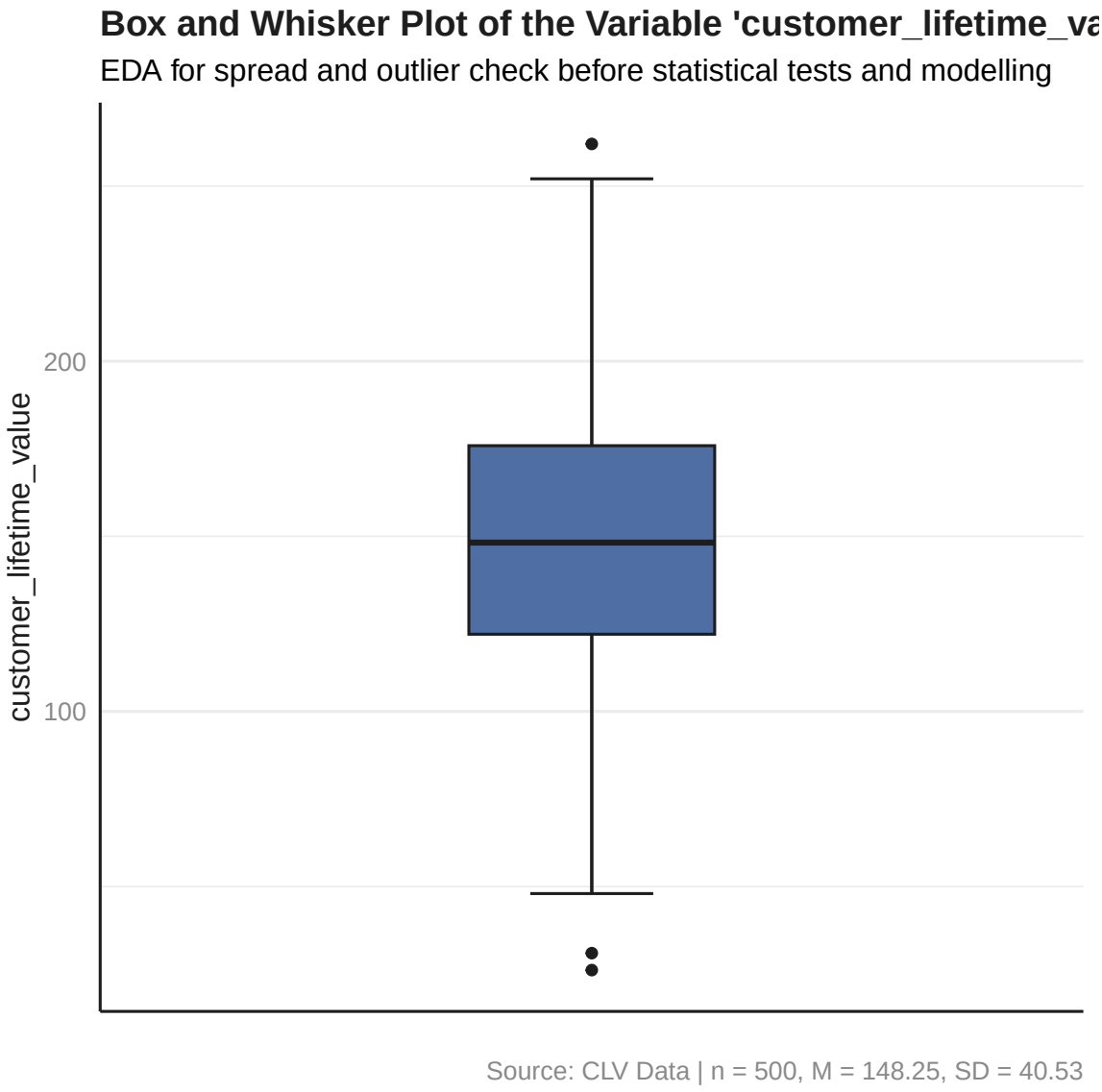
```
# `col_index` identifies which column(s) (by position) are being plotted;
# seq_along() generates 1, 2, ..., ncol(data_frame) automatically,
# so every column in the data frame is covered without manually listing
# positions
col_index <- seq_along(clv_data)
```

```r
# the right-hand `col_index` (the full vector) is evaluated once when the
# loop starts; col_index is then reassigned to one position per iteration.
for (col_index in col_index) {

  # skip silently, with no message/warning, if the position does not exist
  # in the data (out of range) or if the column at that position is not
  # numeric; `next` moves straight to the following value of col_index
  if (col_index > ncol(clv_data) ||
      !is.numeric(clv_data[[col_index]])) {
    next
  }

  col_name <- names(clv_data)[col_index]

  p <- ggplot2::ggplot(clv_data, aes(x = "", y = .data[[col_name]])) +
    stat_boxplot(geom = "errorbar", width = 0.15, color = "#1D1D1D") +
    geom_boxplot(fill = "#4F6EA4", color = "#1D1D1D", width = 0.3) +

    # reduces the default 5% padding on the x and y axes
    # scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
    # scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +
    labs(
      title = paste0("Box and Whisker Plot of the Variable '", col_name, "'"),
      subtitle = paste0("EDA for spread and outlier check before ",
                        "statistical tests and modelling"),
      x = NULL,  # no x-axis label needed
      y = col_name,
      caption = paste0("Source: CLV Data | n = ",
                       nrow(clv_data), ", ",
                       "M = ",
                       sprintf("%.2f",
                               mean(clv_data[[col_name]], na.rm = TRUE)),
                       ", ",
                       "SD = ",
                       sprintf("%.2f",
                               sd(clv_data[[col_name]], na.rm = TRUE)))
    ) +
    theme_minimal(base_size = 12) +

    theme(
      plot.title          = element_text(face = "bold", color = "#1D1D1D"),
      axis.title          = element_text(color = "#1D1D1D"),
```

```
    axis.text           = element_text(color = "#898989"),
    axis.text.x         = element_text(angle = 30, hjust = 1),
    plot.caption         = element_text(color = "#898989"),
    # removes vertical gridlines
    panel.grid.major.x  = element_blank(),
    panel.grid.minor.x  = element_blank(),
    # horizontal gridlines are kept (at major breaks only) since they
    # help the reader judge bar heights against the y-axis scale
    # This is part of maintaining a high data-to-ink ratio in your
    # visualizations as discussed during the theory class.
    # panel.grid.major.y  = element_blank(),
    panel.grid.minor.y  = element_blank(),
    # draws only the x-axis and y-axis lines (the "L" frame)
    axis.line.x         = element_line(color = "#1D1D1D"),
    axis.line.y         = element_line(color = "#1D1D1D")
    # panel.border = element_rect(color = "#1D1D1D", fill = NA,
    #                             linewidth = 0.5)
  ) +
  theme_minimal(base_size = 12) +

  theme(
    plot.title          = element_text(face = "bold", color = "#1D1D1D"),
    axis.title          = element_text(color = "#1D1D1D"),
    axis.text           = element_text(color = "#898989"),
    axis.text.x         = element_text(angle = 30, hjust = 1),
    plot.caption         = element_text(color = "#898989"),
    # removes vertical gridlines
    panel.grid.major.x  = element_blank(),
    panel.grid.minor.x  = element_blank(),
    # horizontal gridlines are kept (at major breaks only) since they
    # help the reader judge bar heights against the y-axis scale
    # This is part of maintaining a high data-to-ink ratio in your
    # visualizations as discussed during the theory class.
    # panel.grid.major.y  = element_blank(),
    # panel.grid.minor.y  = element_blank(),
    # draws only the x-axis and y-axis lines (the "L" frame)
    axis.line.x         = element_line(color = "#1D1D1D"),
    axis.line.y         = element_line(color = "#1D1D1D")
    # panel.border = element_rect(color = "#1D1D1D", fill = NA,
    #                             linewidth = 0.5)
  )
# ggplot objects do not auto-print inside a for-loop, therefore,
```

```
  # each one must be printed explicitly otherwise it will never appear
  print(p)
}
```

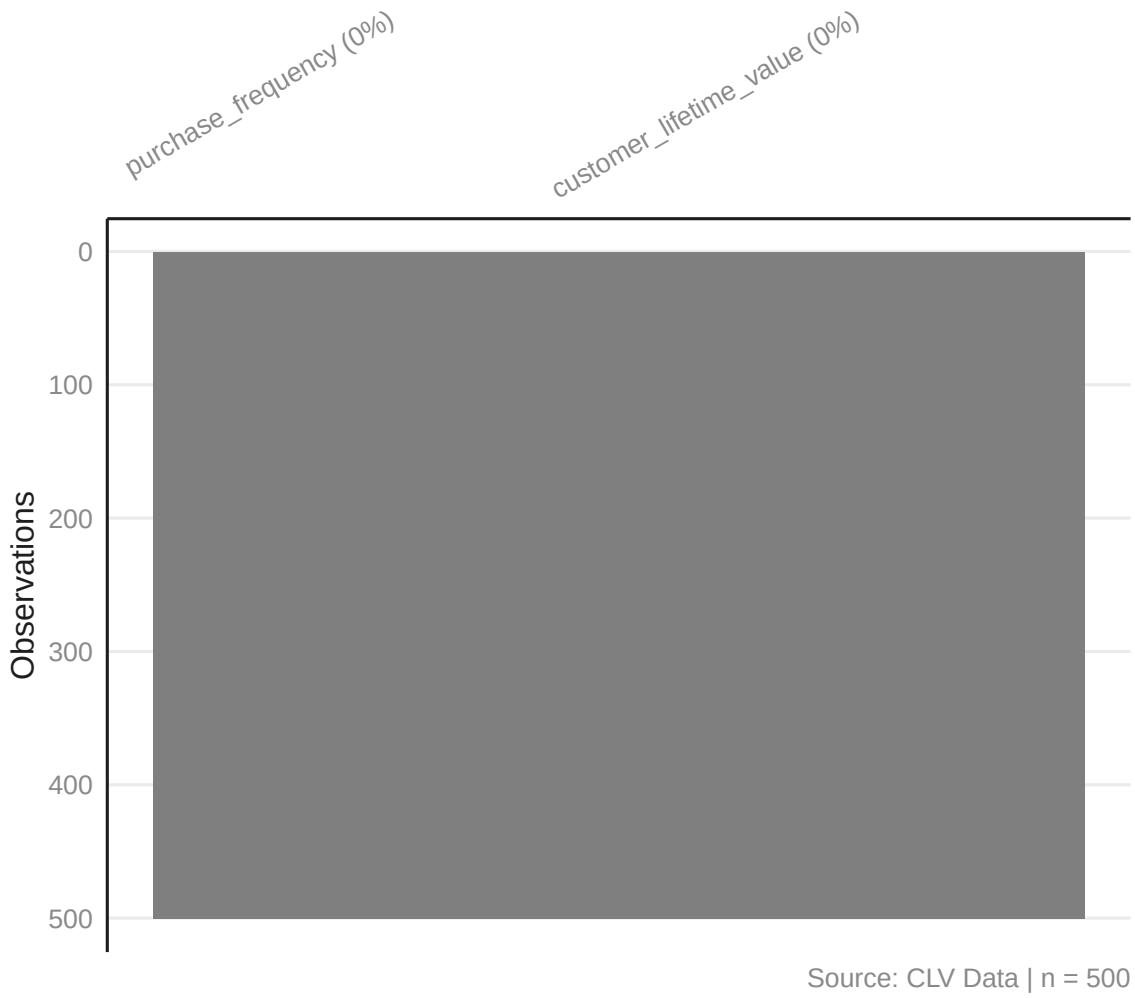





### 4.5.3 Missing Data Plot

```
# `vis_miss()` plots a matrix of observations (rows) by variables (columns),
# shading each cell by whether the value is missing or present
naniar::vis_miss(clv_data) +
  # reduces the default 5% padding on the x and y axes
  # scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
  # scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +
```

```r
  labs(
    title = "Missing Data Overview",
    subtitle = "EDA for missingness pattern before data imputation or data removal",
    caption = paste0("Source: CLV Data | n = ",
                     nrow(clv_data))
  ) +
  # overrides vis_miss()'s default black/grey fill; vis_miss() has no
  # built-in color argument, but since it returns a standard ggplot
  # object, the "fill scale" can be replaced afterward
  scale_fill_manual(values = c("Present" = "#4F6EA4", "Missing" = "#E03C31")) +
  theme_minimal(base_size = 12) +

  theme(
    plot.title          = element_text(face = "bold", color = "#1D1D1D"),
    axis.title          = element_text(color = "#1D1D1D"),
    axis.text           = element_text(color = "#898989"),
    axis.text.x         = element_text(angle = 30, hjust = 1),
    plot.caption         = element_text(color = "#898989"),
    # removes vertical gridlines
    panel.grid.major.x  = element_blank(),
    panel.grid.minor.x  = element_blank(),
    # horizontal gridlines are kept (at major breaks only) since they
    # help the reader judge bar heights against the y-axis scale
    # This is part of maintaining a high data-to-ink ratio in your
    # visualizations as discussed during the theory class.
    # panel.grid.major.y  = element_blank(),
    panel.grid.minor.y  = element_blank(),
    # draws only the x-axis and y-axis lines (the "L" frame)
    axis.line.x         = element_line(color = "#1D1D1D"),
    axis.line.y         = element_line(color = "#1D1D1D")
    # panel.border = element_rect(color = "#1D1D1D", fill = NA,
    #
  )
```



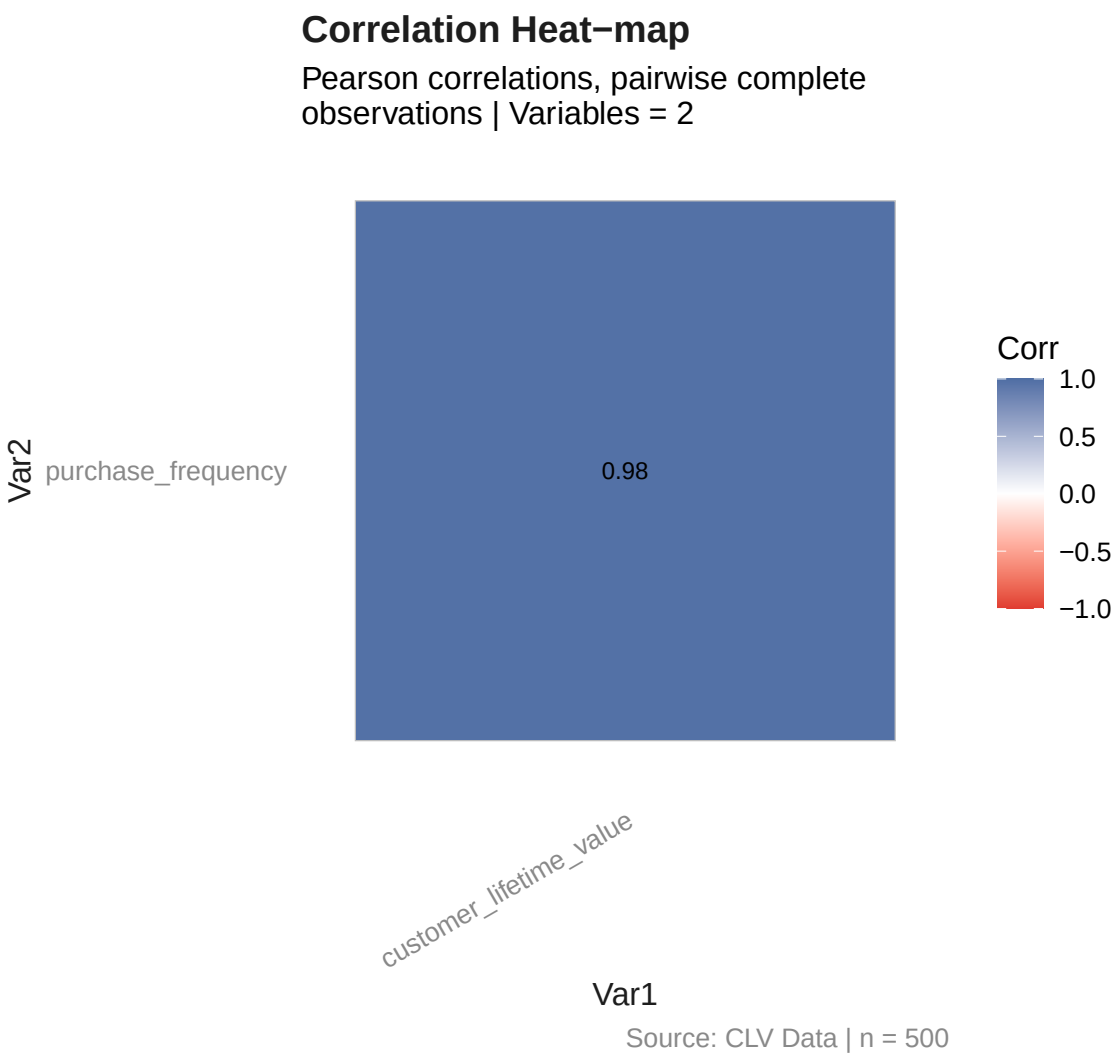

#### 4.5.4 Correlation Plot

```
# Identify all numeric columns in the data frame.
# This mirrors the is.numeric() guard used inside the previouys
# boxplot and histogram loops.
# Non-numeric columns carry no meaningful Pearson correlation and are
# excluded silently before any computation begins, preventing cor() from
# throwing an error.
```

```r
numeric_cols <- names(clv_data)[sapply(clv_data, is.numeric)]

# Guard: cor() requires at least two numeric columns to produce a matrix.
# Fewer than two means there is nothing to correlate. We issue an informative
# message and skip the entire block rather than letting downstream code
# fail with an uninformative error.
if (length(numeric_cols) < 2) {
  message(
    "Correlation plot skipped: fewer than two numeric columns found."
  )
} else {

  # Subset to numeric columns only before passing to cor().

  # When you subset a data frame with [, singlecolumnname], R silently drops
  # the data frame structure and returns a plain vector instead.

  # A vector has no columns, so cor() would immediately fail.
  # drop = FALSE suppresses that behaviour, forcing R to always return a data
  # frame regardless of how many columns are selected.

  # pairwise.complete.obs means each pair of variables uses all rows where
  # both values are non-missing.
  corr_matrix <- cor(
    clv_data[, numeric_cols, drop = FALSE],
    use = "pairwise.complete.obs"
  )

  # ggcorrplot() is a ggplot2 wrapper purpose-built for correlation matrices.
  # It returns a standard ggplot2 object, so the full suite of labs(),
  # theme_minimal(), and theme() overrides can be appended with `+`.
  #
  # hc.order = TRUE  reorders variables by hierarchical clustering so that
  #                  strongly correlated pairs appear adjacent; this makes
  #                  it easier to read than the default column order.
  # type = "lower"   shows only the lower triangle of the symmetric matrix,
  #                  thus removing redundant information.
  # lab = TRUE       prints the rounded correlation coefficient inside each
  #                  tile, allowing precise reading without relying on the
  #                  colour scale alone.
  # lab_size = 3     keeps labels legible even when many variables are present.
  p <- ggcorrplot::ggcorrplot(
```

```
  corr_matrix,
  hc.order = TRUE,
  type     = "lower",
  lab      = TRUE,
  lab_size = 3,

  # Diverging colour palette: negative correlations are mapped to the red
  # anchor, zero (no linear relationship) to neutral white, and positive
  # correlations to blue.
  # This preserves colour-semantic meaning (direction matters for
  # correlation in a way it does not for boxplot fills) while keeping the
  # positive anchor consistent with the house style.
  colors = c("#E03C31", "#FFFFFF", "#4F6EA4")  # negative → neutral → positive
) +

  # reduces the default 5% padding on the x and y axes
  # scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
  # scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +

  labs(
    title    = "Correlation Heat-map",
    subtitle = paste0(
      "Pearson correlations, pairwise complete\n",
      "observations | Variables = ", length(numeric_cols)
    ),
    # The caption follows the same n / M / SD convention used in the
    # boxplots; here n refers to the number of observations in the data,
    # not the number of variable pairs.
    caption  = paste0("Source: CLV Data | n = ", nrow(clv_data))
  ) +
  theme_minimal(base_size = 12) +

  theme(
    plot.title          = element_text(face = "bold", color = "#1D1D1D"),
    axis.title          = element_text(color = "#1D1D1D"),
    axis.text           = element_text(color = "#898989"),
    axis.text.x         = element_text(angle = 30, hjust = 1),
    plot.caption         = element_text(color = "#898989"),
    # removes vertical gridlines
    panel.grid.major.x  = element_blank(),
    panel.grid.minor.x  = element_blank(),
    # horizontal gridlines are kept (at major breaks only) since they
```

```
      # help the reader judge bar heights against the y-axis scale
      # This is part of maintaining a high data-to-ink ratio in your
      # visualizations as discussed during the theory class.
      panel.grid.major.y  = element_blank(),
      panel.grid.minor.y  = element_blank(),
      # draws only the x-axis and y-axis lines (the "L" frame)
      # axis.line.x         = element_line(color = "#1D1D1D"),
      # axis.line.y         = element_line(color = "#1D1D1D")
      # panel.border = element_rect(color = "#1D1D1D", fill = NA,
      #                             linewidth = 0.5)
    )
  # ggplot objects do not auto-print inside a for-loop, therefore,
  # each one must be printed explicitly otherwise it will never appear
  print(p)
}
```



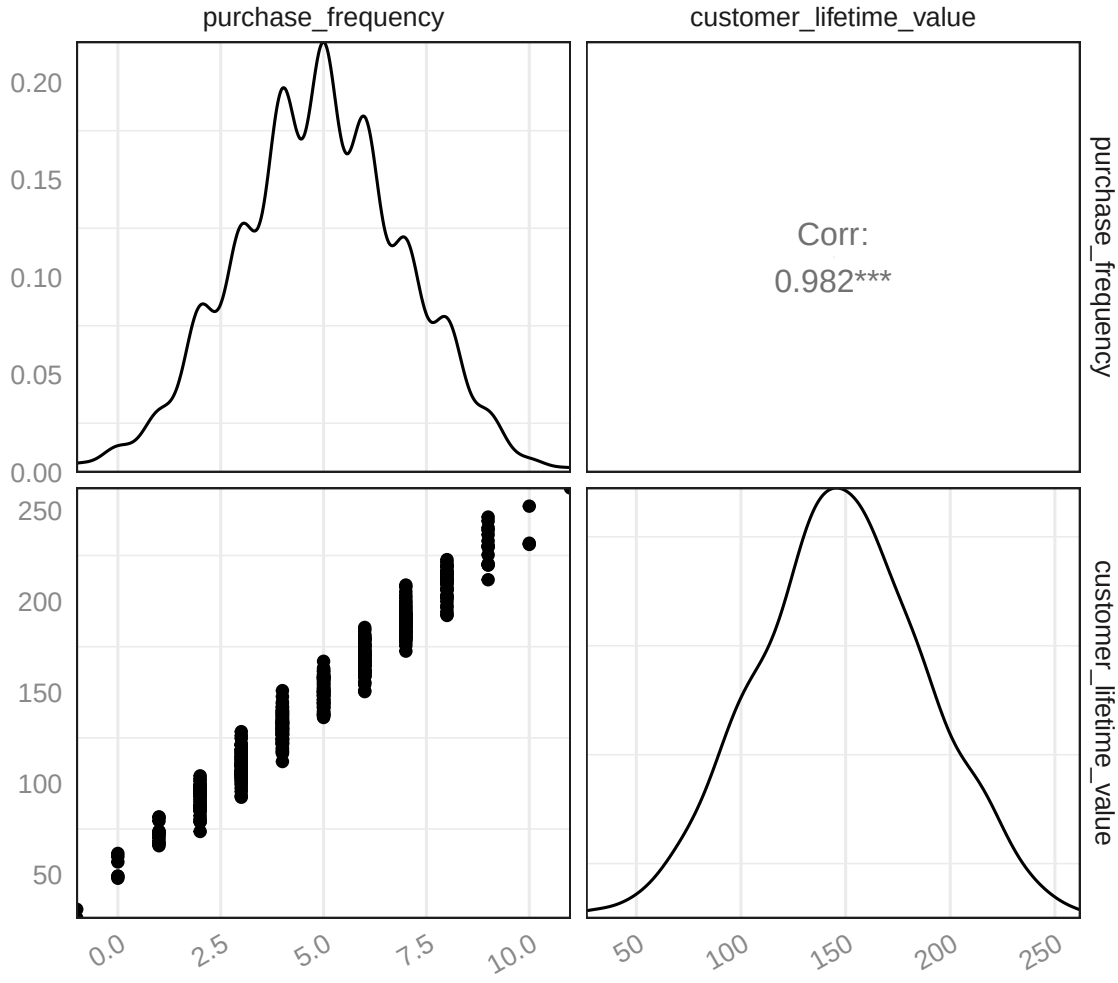

#### 4.5.5 Scatter Plot

```
GGally::ggpairs(clv_data) +

  # reduces the default 5% padding on the x and y axes
  scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
  scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +
  labs(
```

```
  title = "Pairwise Relationships between All Variables",
  subtitle = paste0("EDA for pairwise correlations and distributions\n",
                    "before statistical tests and before modelling.")
) +
theme_minimal(base_size = 12) +

theme(
  plot.title          = element_text(face = "bold", color = "#1D1D1D"),
  axis.title          = element_text(color = "#1D1D1D"),
  axis.text           = element_text(color = "#898989"),
  axis.text.x         = element_text(angle = 30, hjust = 1),
  plot.caption         = element_text(color = "#898989"),
  # removes vertical gridlines
  # panel.grid.major.x  = element_blank(),
  panel.grid.minor.x  = element_blank(),
  # horizontal gridlines are kept (at major breaks only) since they
  # help the reader judge bar heights against the y-axis scale
  # This is part of maintaining a high data-to-ink ratio in your
  # visualizations as discussed during the theory class.
  panel.grid.major.y  = element_blank(),
  # panel.grid.minor.y  = element_blank(),
  # draws only the x-axis and y-axis lines (the "L" frame)
  # axis.line.x         = element_line(color = "#1D1D1D"),
  # axis.line.y         = element_line(color = "#1D1D1D")
  panel.border = element_rect(color = "#1D1D1D", fill = NA,
                              linewidth = 0.5)
)
```

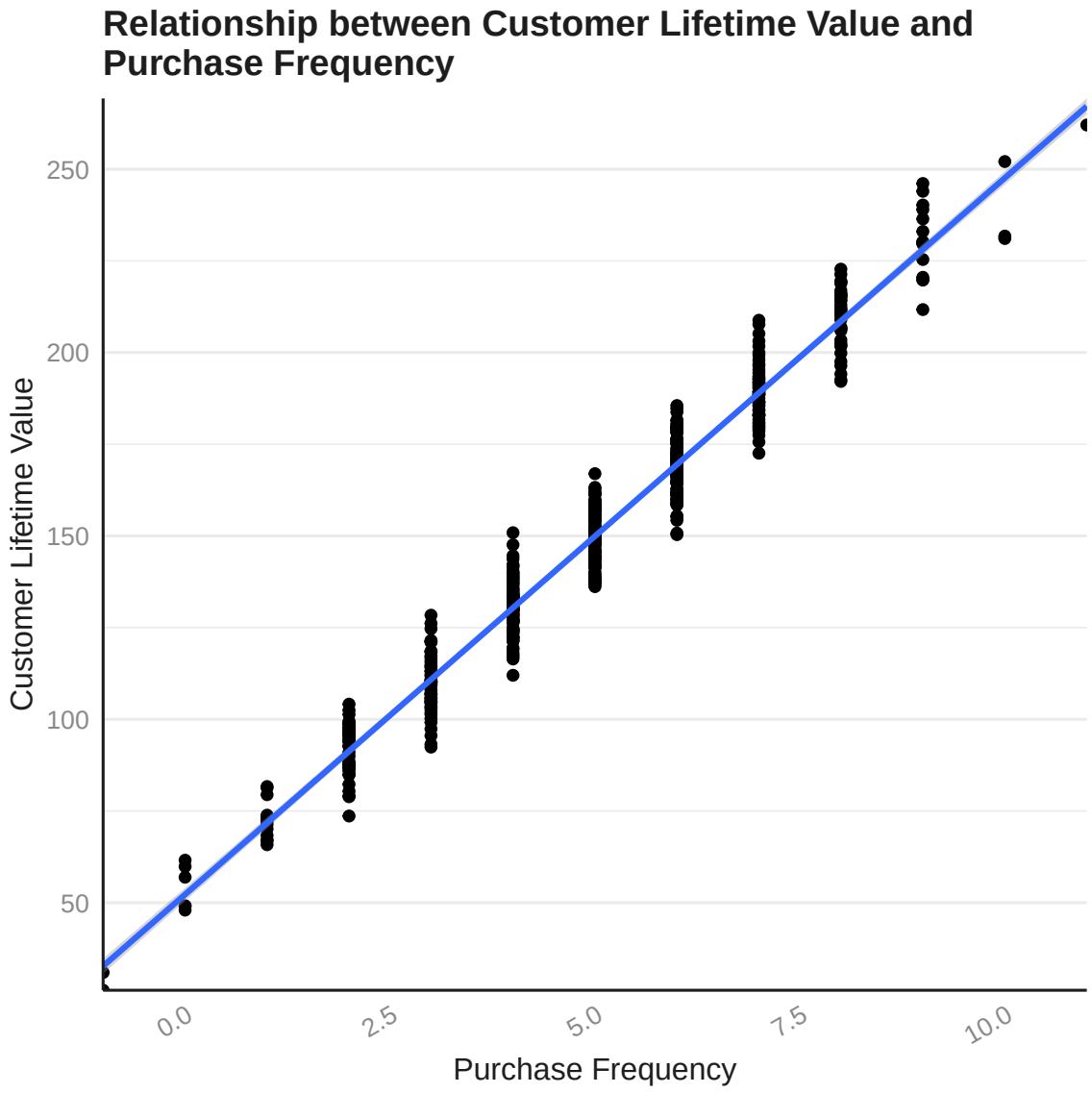
## Pairwise Relationships between All Variables

EDA for pairwise correlations and distributions
before statistical tests and before modelling.



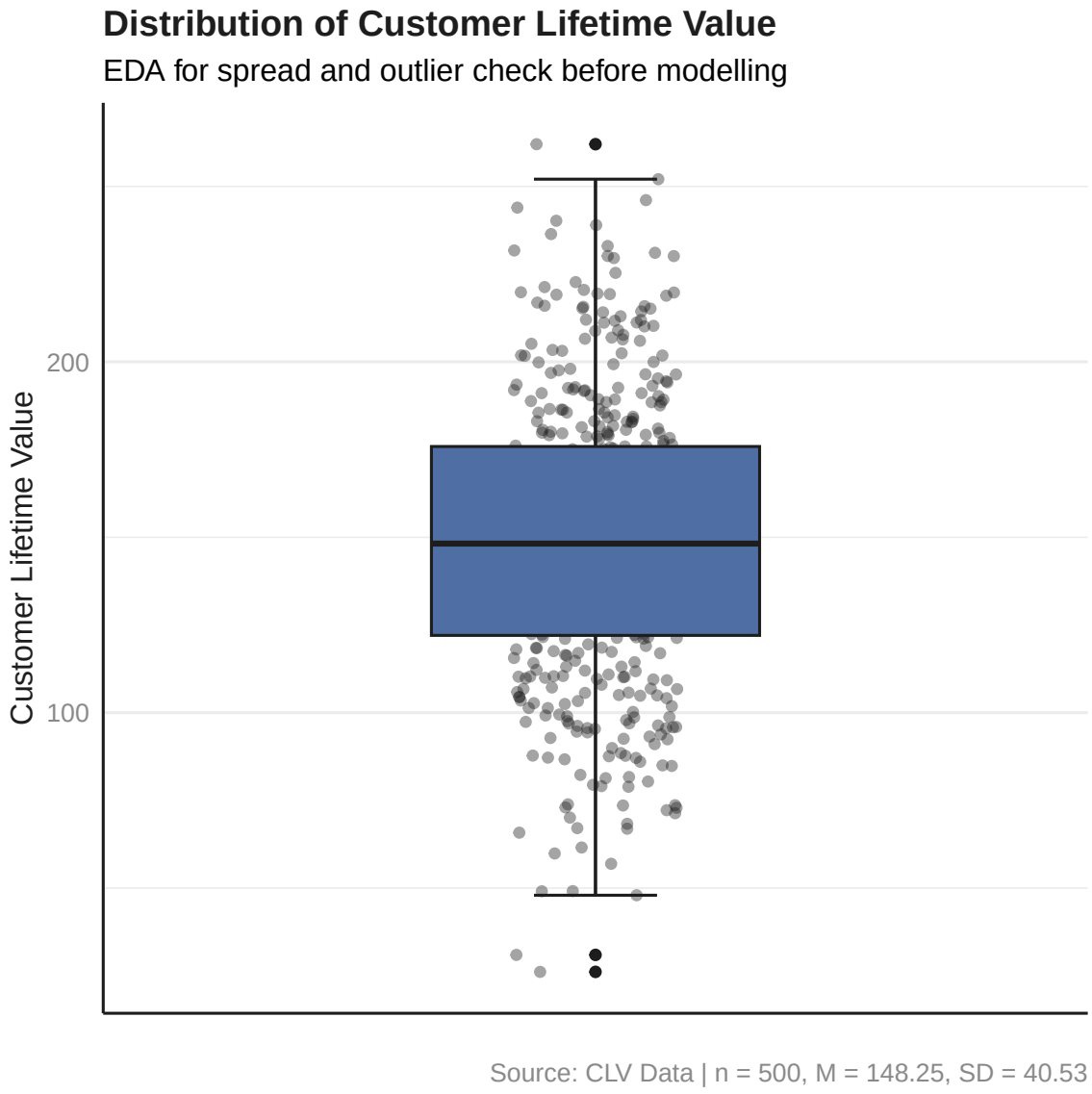

```
pacman::p_load("ggplot2")
ggplot(clv_data,
       # It is possible to plot specific variables
       aes(x = purchase_frequency, y = customer_lifetime_value)) +
  geom_point() +
  geom_smooth(method = lm) +

  # reduces the default 5% padding on the x and y axes
  scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
```

```r
  scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +

  labs(
    title = paste0("Relationship between Customer Lifetime Value and\n",
    "Purchase Frequency"),
    x = "Purchase Frequency",
    y = "Customer Lifetime Value"
  ) +
  theme_minimal(base_size = 12) +

  theme(
    plot.title          = element_text(face = "bold", color = "#1D1D1D"),
    axis.title          = element_text(color = "#1D1D1D"),
    axis.text           = element_text(color = "#898989"),
    axis.text.x         = element_text(angle = 30, hjust = 1),
    plot.caption         = element_text(color = "#898989"),
    # removes vertical gridlines
    panel.grid.major.x  = element_blank(),
    panel.grid.minor.x  = element_blank(),
    # horizontal gridlines are kept (at major breaks only) since they
    # help the reader judge bar heights against the y-axis scale
    # This is part of maintaining a high data-to-ink ratio in your
    # visualizations as discussed during the theory class.
    # panel.grid.major.y  = element_blank(),
    # panel.grid.minor.y  = element_blank(),
    # draws only the x-axis and y-axis lines (the "L" frame)
    axis.line.x         = element_line(color = "#1D1D1D"),
    axis.line.y         = element_line(color = "#1D1D1D")
    # panel.border = element_rect(color = "#1D1D1D", fill = NA,
    #                             linewidth = 0.5)
  )
```




```
ggplot2::ggplot(clv_data,
                aes(x = "", y = customer_lifetime_value)) +
  stat_boxplot(geom = "errorbar", width = 0.15, color = "#1D1D1D") +
  # layering the raw data points underneath the box with geom_jitter()
  # shows both the summary statistics and the actual spread and sample size,
  # which a box alone hides
  geom_jitter(width = 0.1, alpha = 0.4, color = "#1D1D1D", size = 1.5) +
  geom_boxplot(fill = "#4F6EA4", color = "#1D1D1D", width = 0.4) +
```

```r
  # reduces the default 5% padding on the x and y axes
  # scale_x_continuous(expand = expansion(mult = c(0.00, 0.00))) +
  # scale_y_continuous(expand = expansion(mult = c(0.00, 0.00))) +

  labs(
    title = "Distribution of Customer Lifetime Value",
    subtitle = "EDA for spread and outlier check before modelling",
    x = NULL,
    y = "Customer Lifetime Value",
    caption = paste0("Source: CLV Data | n = ", nrow(clv_data), ", ",
                     "M = ", sprintf("%.2f", mean(clv_data$customer_lifetime_value,
                                                  na.rm = TRUE)), ", ",
                     "SD = ", sprintf("%.2f", sd(clv_data$customer_lifetime_value,
                                                 na.rm = TRUE)))
  ) +
  theme_minimal(base_size = 12) +

  theme(
    plot.title          = element_text(face = "bold", color = "#1D1D1D"),
    axis.title          = element_text(color = "#1D1D1D"),
    axis.text           = element_text(color = "#898989"),
    axis.text.x         = element_text(angle = 30, hjust = 1),
    plot.caption         = element_text(color = "#898989"),
    # removes vertical gridlines
    panel.grid.major.x  = element_blank(),
    panel.grid.minor.x  = element_blank(),
    # horizontal gridlines are kept (at major breaks only) since they
    # help the reader judge bar heights against the y-axis scale
    # This is part of maintaining a high data-to-ink ratio in your
    # visualizations as discussed during the theory class.
    # panel.grid.major.y  = element_blank(),
    # panel.grid.minor.y  = element_blank(),
    # draws only the x-axis and y-axis lines (the "L" frame)
    axis.line.x         = element_line(color = "#1D1D1D"),
    axis.line.y         = element_line(color = "#1D1D1D")
    # panel.border = element_rect(color = "#1D1D1D", fill = NA,
    #                             linewidth = 0.5)
  )
```




## 5 Statistical Test

We then apply simple linear regression as a statistical test for regression.

```
slr_test <- lm(customer_lifetime_value ~ purchase_frequency, data = clv_data)
```

View the summary of the model.

```r
summary(slr_test)
```

```
#>
#> Call:
#> lm(formula = customer_lifetime_value ~ purchase_frequency, data = clv_data)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -19.1176  -5.6169  -0.0491   5.6618  20.4837
#>
#> Coefficients:
#>                    Estimate Std. Error t value Pr(>|t|)
#> (Intercept)         52.2538     0.9042   57.79   <2e-16 ***
#> purchase_frequency  19.5356     0.1700  114.91   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 7.734 on 498 degrees of freedom
#> Multiple R-squared:  0.9637, Adjusted R-squared:  0.9636
#> F-statistic: 1.32e+04 on 1 and 498 DF,  p-value: < 2.2e-16
```

The confidence level represents the degree of certainty that a confidence interval contains the true population parameter. For example, a 95% confidence level means that if you were to take many random samples and compute the confidence interval from each, about 95% of those intervals would contain the true population value, while about 5% would not.

To obtain a 95% confidence interval:

```r
confint(slr_test, level = 0.95)
```

```
#>                       2.5 %   97.5 %
#> (Intercept)        50.47731 54.03036
#> purchase_frequency 19.20159 19.86965
```

## 6 Diagnostic EDA (Model Diagnostics)

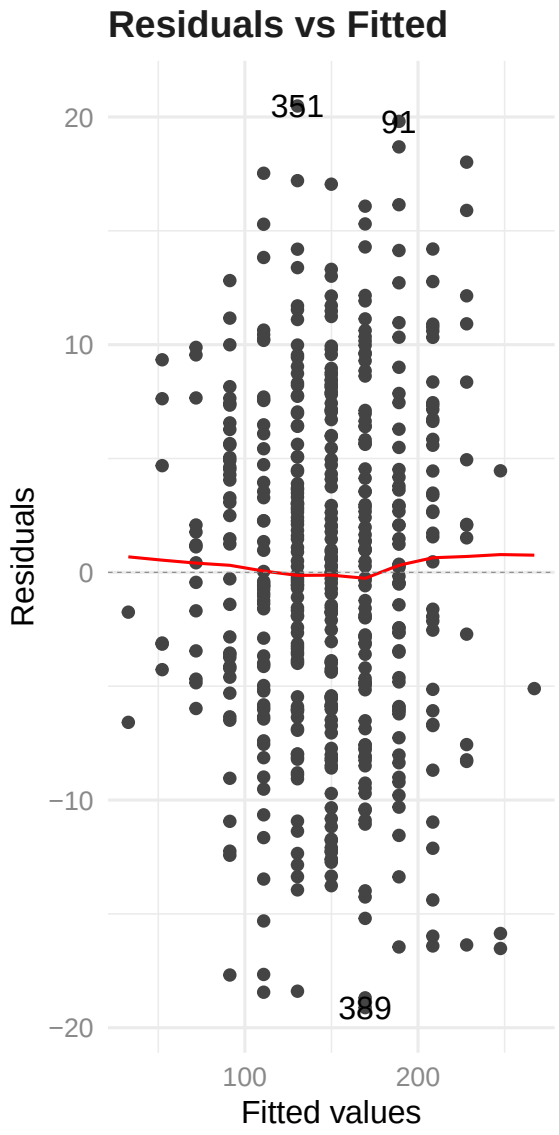
Diagnostic Exploratory Data Analysis (EDA) is performed to verify that the assumptions underlying the regression model are satisfied. Confirming that these assumptions hold ensures that the statistical tests used in the model are valid for the data, thus reducing the risk of drawing incorrect or misleading conclusions in your data analysis.

## 6.1 Test of Linearity

The test of linearity is used to assess whether the relationship between the dependent variable and the independent variable(s) is linear. This is necessary given that linearity is one of the key assumptions of statistical tests of regression and verifying it is crucial for ensuring the validity of the model's estimates and predictions.

A plot of the residuals versus the fitted values enables us to test for linearity. For the model to pass the test of linearity, there should be no pattern in the distribution of residuals and the residuals should be randomly placed around the 0.0 residual line.

```
pacman::p_load("ggfortify")
autoplot(slr_test, which = 1, smooth.colour = "red") +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", color = "#1D1D1D"),
        axis.text  = element_text(color = "#898989"))
```




## 6.2 Test of Independence of Errors (Autocorrelation)

This test is necessary to confirm that each observation is independent of the other. It helps to identify autocorrelation that is introduced when the data is collected over a close period of time or when one observation is related to another observation. Autocorrelation leads to underestimated standard errors and inflated t-statistics. It can also make findings appear more significant than they actually are. The "Durbin-Watson Test" can be used as a test of independence of errors (test of autocorrelation). A Durbin-Watson statistic close to 2

suggests no autocorrelation, while values approaching 0 or 4 indicate positive or negative autocorrelation, respectively.

For the Durbin-Watson test:

- The null hypothesis, $H_0$, is that there is no autocorrelation (no autocorrelation = there is no correlation between residuals across time or across observations).
- The alternative hypothesis, $H_a$, is that there is autocorrelation (autocorrelation = there is a correlation between residuals across time or across observations)

If the p-value of the Durbin-Watson statistic is greater than 0.05 then there is no evidence to reject the null hypothesis that "there is no autocorrelation".

```
pacman::p_load("lmtest")
dwtest(slr_test)
```

```
#>
#>  Durbin-Watson test
#>
#> data:  slr_test
#> DW = 1.9104, p-value = 0.1573
#> alternative hypothesis: true autocorrelation is greater than 0
```

The results show a p-value of >.05 (and a DW statistic of 1.91), therefore, the test of independence of errors around the regression line passes, i.e., there is no autocorrelation. In other words, there is no evidence to reject the null hypothesis that states that, "there is no aurocorrelation".

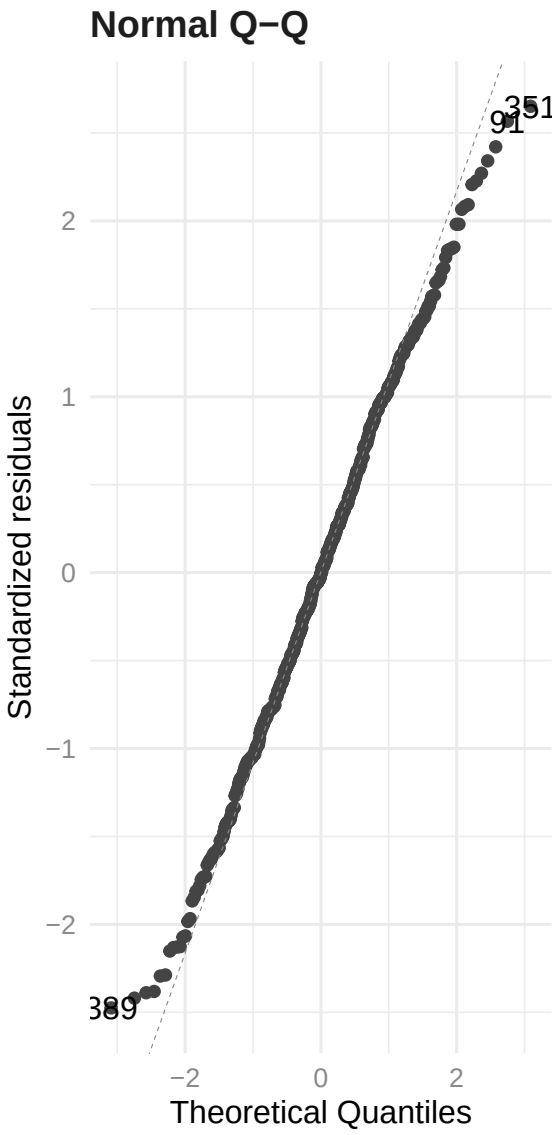
## 6.3 Test of Normality of the Distribution of the Errors

The test of normality of the distribution of the errors assesses whether the errors (residuals) are approximately normally distributed, i.e., most errors are close to zero and large errors are rare. A Q-Q plot can be used to conduct the test of normality.

A Q-Q plot is a scatterplot of the quantiles of the errors against the quantiles of a normal distribution. Quantiles are statistical values that divide a dataset or probability distribution into equal-sized intervals. They help in understanding how data is distributed by marking specific points that separate the data into groups of equal size. Examples of quantiles include: quartiles (4 equal parts), percentiles (100 equal parts), deciles (10 equal parts), etc.

If the points in the Q-Q plot fall along a straight line, then the normality assumption is satisfied. If the points in the Q-Q plot do not fall along a straight line, then the normality assumption is not satisfied.

```r
autoplot(slr_test, which = 2, smooth.colour = NA) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", color = "#1D1D1D"),
        axis.text  = element_text(color = "#898989"))
```



## 6.4 Test of Homoscedasticity

Homoscedasticity requires that the spread of residuals should be constant across all levels of the independent variable. A scale-location plot (a.k.a. spread-location plot) can be used to conduct a test of homoscedasticity.

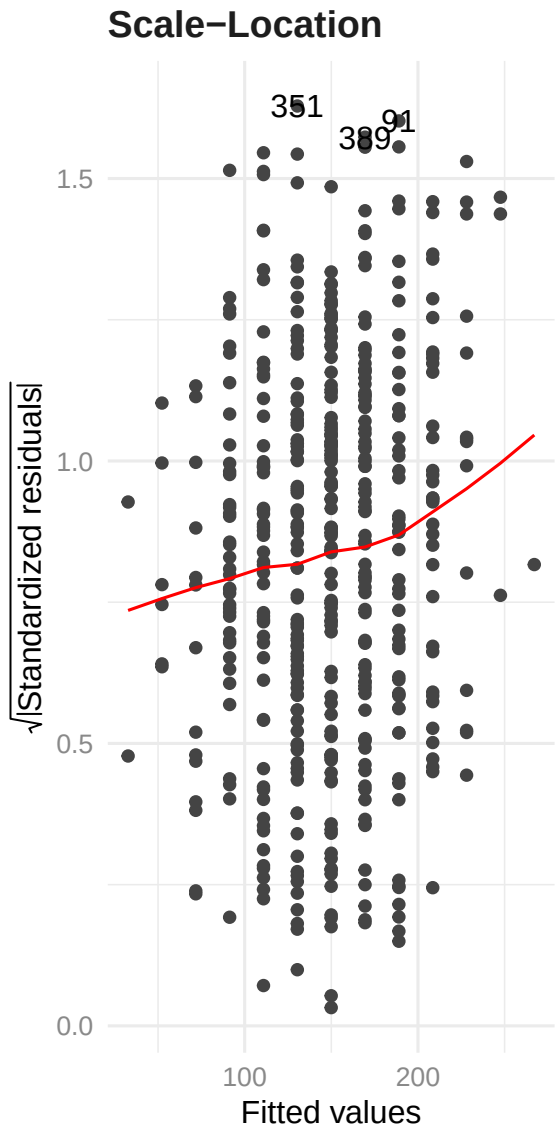
The x-axis shows the fitted (predicted) values from the model and the y-axis shows the square root of the standardized residuals. The red line is added to help visualize any patterns.

In a model with homoscedastic errors (equal variance across all predicted values):

- Points should be randomly scattered around a horizontal line
- The smooth line should be approximately horizontal
- The vertical spread of points should be roughly equal across all fitted values
- No obvious patterns, funnels, or trends should be visible

Points forming a cone shape that widens from left to right suggests heteroscedasticity with increasing variance for larger fitted values.

```
autoplot(slr_test, which = 3, smooth.colour = "red") +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", color = "#1D1D1D"),
        axis.text  = element_text(color = "#898989"))
```




**Breusch-Pagan Test**

The Breusch-Pagan Test can also be used in addition to the visual inspection of a Scale-Location plot.

Formally:

- Null hypothesis (H ): The residuals are homoscedastic (equal variance).
- Alternative hypothesis (H ): The residuals are heteroscedastic (non-constant variance).

p-Value:

- p-value  0.05: Fail to reject H  → no evidence of heteroscedasticity → good, model passes.
- p-value < 0.05: Reject H  → evidence of heteroscedasticity → bad, model fails.

Interpretation: If the p-value is less than 0.05, then we reject the null hypothesis that states that "the residuals are homoscedastic"

With a p-value < 0.05, there is statistically significant evidence of heteroscedasticity in the residuals in this case (which is bad).

```
pacman::p_load("lmtest")
lmtest::bptest(slr_test)
```

```
#>
#>  studentized Breusch-Pagan test
#>
#> data:  slr_test
#> BP = 8.9114, df = 1, p-value = 0.002834
```

## 6.5 Quantitative Validation of Assumptions

The graphical representations of the various tests of assumptions should be accompanied by quantitative values. The `gvlma` package (Global Validation of Linear Models Assumptions) is useful for this purpose.

```
pacman::p_load("gvlma")
gvlma_results <- gvlma(slr_test)
summary(gvlma_results)
```

```
#>
#> Call:
#> lm(formula = customer_lifetime_value ~ purchase_frequency, data = clv_data)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -19.1176  -5.6169  -0.0491   5.6618  20.4837
#>
#> Coefficients:
#>                    Estimate Std. Error t value Pr(>|t|)
#> (Intercept)         52.2538     0.9042   57.79   <2e-16 ***
#> purchase_frequency  19.5356     0.1700  114.91   <2e-16 ***
```

```
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 7.734 on 498 degrees of freedom
#> Multiple R-squared:  0.9637, Adjusted R-squared:  0.9636
#> F-statistic: 1.32e+04 on 1 and 498 DF,  p-value: < 2.2e-16
#>
#>
#> ASSESSMENT OF THE LINEAR MODEL ASSUMPTIONS
#> USING THE GLOBAL TEST ON 4 DEGREES-OF-FREEDOM:
#> Level of Significance =  0.05
#>
#> Call:
#>  gvlma(x = slr_test)
#>
#>                      Value p-value                Decision
#> Global Stat        5.08943 0.27824 Assumptions acceptable.
#> Skewness           0.03973 0.84201 Assumptions acceptable.
#> Kurtosis           3.61252 0.05735 Assumptions acceptable.
#> Link Function      0.01459 0.90385 Assumptions acceptable.
#> Heteroscedasticity 1.42258 0.23298 Assumptions acceptable.
```

# 7 Interpretation of the Results

We can interpret the results of the statistical test with more confidence if the tests of assumptions are successful. The presentation of the results and its subsequent interpretation is based on the following notes.

**t-Statistic t(d.f.):** It quantifies how many standard errors the estimated coefficient deviates from zero. A larger t-value (e.g., >2) indicates stronger evidence against the null hypothesis (i.e., that the coefficient is zero). The t-statistic has its corresponding p-value such that a p-value $< .05$ implies a statistically significant t-statistic.

**Degrees of Freedom (d.f.):** Degrees of freedom refers to the number of values in a calculation that are free to vary. It is essentially a measure of how much independent information is available for estimating a statistical parameter.

For example: Imagine you need to calculate the average height of 5 people, and you know the sum of all their heights is 340 inches. If you know the heights of 4 of these people (65, 70, 68, and 72 inches), you can automatically determine the height of the fifth person without measuring them: 340 - (65 + 70 + 68 + 72) = 65 inches In this example, even though there are 5 people, you only have 4 degrees of freedom because once you know 4 heights and the total, the 5th height is no longer "free to vary" – it is determined by the other values.

### F-Statistic

**F(d.f. in numerator, d.f. in denominator):** The numerator degree of freedom corresponds to the number of predictor variables, while the denominator degree of freedom is derived from the total number of observations minus the number of predictors and then minus 1 for the intercept.

The F-test in regression evaluates whether the variance explained by the model is significantly greater than the unexplained variance (error). Think of the F-statistic as a ratio of "signal" (useful prediction) to "noise" (unexplained variation). The higher this ratio, the more confident you can be that your model is capturing something real. The larger the F-Statistic, the better the model's performance.

Also, a low p-value of the F-statistic (any p-value $< .05$ is considered low) indicates that the overall regression model **is statistically significant**.

### Coefficient of Determination ($R^2$)

The R-squared value represents the proportion of the total variation in the dependent variable that can be attributed to or explained by the independent variable. An R-squared of 0.96 indicates that approximately 96% of the variability in the dependent variable can be explained by its linear relationship with the independent variable. An R-squared value approaching 1 signifies that the regression line closely aligns with the observed data points.

**Multiple R-squared:** Measures the proportion of variance in the dependent variable explained by the independent variable (e.g., Multiple $R^2 = 0.6$ means 60% of sales variance is explained by advertisement expenditure). The multiple R-squared value always increases (or at least never decreases) when you add more independent variables.

**Adjusted R-squared:** Also measures the proportion of variance in the dependent variable explained by the independent variable, however, it introduces a penalty based on the number of independent variables relative to the sample size.

The difference between multiple R-squared and adjusted R-squared is negligible in cases where there is only 1 independent variable.

### Residual Standard Error

The residual standard error quantifies the average magnitude of the errors (residuals), which are the discrepancies between the observed values in the dataset and the values predicted by the regression model. It represents the standard deviation of the data points around the regression line. For example, a residual standard error of 7.73 indicates that, on average, the model's predicted value of the dependent variable deviates from the actual observed value by approximately 7.73 units.

A smaller residual standard error implies that the data points are more tightly clustered around the regression line, indicating a more precise model.

### Confidence Interval

A 95% confidence interval (CI) for a parameter—such as a regression coefficient—provides a range that, under repeated sampling, would contain the true (but unknown) population parameter 95% of the time. Analogy: Imagine shooting arrows at a target. If you drew a circle around where 95% of your arrows landed, that circle is like a confidence interval—it captures the region in which your "shots" (i.e., estimates from different samples) tend to fall.

**Uncertainty quantification:** A CI communicates your estimate's precision—narrower intervals imply more precise estimates (often due to larger samples or less variability), whereas wider intervals indicate greater uncertainty about the true value.

**Academic Reporting (based on the APA 7th Edition Style)**

Below are some key considerations to note when reporting statistical analysis using the APA style:

1. The type of statistical test must be stated.
2. Although not mandatory, the dependent variable is usually stated first followed by the independent variable when describing relationships, e.g., "…to examine whether advertising expenditures on YouTube, TikTok, and Facebook collectively predict Sales" such that Sales is the dependent variable that depends on advertising expenditures on YouTube, TikTok, and Facebook.
3. Test statistic and parameters: Report the appropriate test statistic ($t$-Statistic, $F$-Statistic, $\chi^2$ , etc.) with the degrees of freedom in parentheses. The italicized statistical symbol is immediately followed by the degrees of freedom in parentheses without a space, e.g., $t(498)$ and not t (498).
4. Exact p-values: Report exact p-values, when possible (e.g., $p = .032$), unless they are less than .001, then report as $p < .001$.
5. Effect sizes: Include appropriate effect size measures (e.g., $R^2$) to indicate practical significance.
6. Standard errors: Report standard errors of estimates when relevant. The standard error tells you how much your estimate might vary if you were to repeat your study with different random samples from the same population. A smaller standard error indicates a more precise estimate.
7. Confidence Intervals (CI): The confidence level should be clearly stated whenever you report point estimates (e.g., means, regression coefficients, correlations, etc.). The 95% confidence interval is the most common, and if another level is used (e.g., 90% CI, 99% CI), it should be explicitly mentioned. Confidence intervals are typically enclosed in square brackets [], with the lower and upper limits separated by a comma. For example: 95% CI [-.03, .04]. They are usually reported directly after the statistic they describe, often within the same sentence or in parentheses.

8. Two decimal places: Report to two decimal places, except p-values which may need three or more decimal places.
9. Descriptive statistics: Report relevant means, standard deviations, and sample sizes, e.g., The sample size included 500 observations (M = 25.43, SD = 4.62).
10. Italicize statistical symbols: Use italics for statistical symbols ($t$, $F$, $p$, etc.) but not for Greek letters ( $\mu$, $\sigma$, $\alpha$), subscripts, or parenthetical information, e.g., $R^2 = .45$, $F(2, 97) = 15.62$, $p < .001$, The participants ($N = 120$) had an average score ($M = 25.43$, $SD = 4.62$) on the cognitive test.

Further reading: https://apastyle.apa.org/jars

## 7.1 Limitations and Diagnostic Findings

The model employed is a simple linear regression, which only considers the linear relationship between purchase frequency and CLV. Other potentially influential factors that are not included in this model could also play a significant role in determining CLV, e.g., the average monetary value of each purchase.

## 7.2 Academic Statement (APA)—Academic-Ready Language

A simple linear regression was conducted on data from 500 observations (N = 500) to examine the relationship between customer lifetime value (CLV) and purchase frequency. The results indicated that purchase frequency significantly predicted CLV, $\beta = 19.54$, 95% CI [19.20, 19.87], SE = 0.17, $t(498) = 114.91$, $p < .001$. The model explained 96.37% of the variance in CLV ($R^2 = .96$, $F(1, 498) = 13{,}200$, $p < .001$). For every unit increase in purchase frequency, CLV increased by approximately 19.54 units. The intercept was 52.25, 95 % CI [50.48, 54.03], and the residual standard error was 7.73, indicating strong predictive accuracy.

## 7.3 Business Analysis—Boardroom-Ready Language

The strength of the relationship highlights the critical importance of customer retention. Initiatives that effectively encourage repeat purchases appear to be a primary driver of customer lifetime value based on this analysis. This understanding can guide the allocation of resources towards strategies that foster customer loyalty and encourage repeat business.

# 8 Rendering the Document

The "Render" button in RStudio (or `quarto render` in the terminal) can be used to convert this Quarto document into either a:

1. HTML document that can be opened using a browser
2. Word Document
3. PDF document

The conversion to PDF requires the installation of the following free software:

- For Windows: MiKTeX - https://miktex.org/download
- For MacOS: MacTeX - https://www.tug.org/mactex/mactex-download.html
- For Linux: TeX Live - https://www.tug.org/texlive/quickinstall.html

Also, you need to install the `tinytex` package. The `tinytex` package helps RStudio to find and use MikTex, MacTeX, or TeXLive. Execute the following **in the console section of RStudio** to install TinyTex:

```
install.packages("tinytex")
```

```
tinytex::install_tinytex()
```

If you are using MiKTeX for Windows, you should also enable the installation of packages on-the-fly. This is found in "Settings > General > Package Installation"

Lastly, ensure that `pdf-engine: xelatex` is set in the YAML front matter under `format: pdf:`, as shown at the top of this document.

# 9 References and Further Reading

American Psychological Association. (2025, February). *Journal Article Reporting Standards (JARS)*. APA Style. Retrieved April 28, 2025, from https://apastyle.apa.org/jars

Hodeghatta, U. R., & Nayak, U. (2023). *Practical Business Analytics Using R and Python: Solve Business Problems Using a Data-driven Approach* (2nd ed.). Apress. https://link.springer.com/book/10.1007/978-1-4842-8754-5